

Tokens: How Language Becomes Computation

FRIDAY AI Education — Episode 004 · Printable Companion

Abstract

A language model does not read. It does not see words, sentences, or paragraphs the way you do on this page. Before any model can predict, summarize, or reason over text, that text is chopped into small units called **tokens** and converted into numbers. Everything downstream — what the model can hold in mind, what it costs you to run, why the same information formatted two different ways produces two different answers — flows from this one mechanical step.

This paper explains tokenization from first principles: what a token is, how text gets cut into tokens, why the vocabulary that does the cutting matters, and how the finite **context length** of a model turns "just give it all the data" into an engineering decision with a price tag. The goal is not to make you a tokenizer engineer. It is to give you a correct mental model so that when you design a knowledge base, ingest a 300-page PDF, or wonder why a clean prompt beat a messy one, you understand the floor you are standing on.

1. Why this is the right place to start

You are building FRIDAY to do real work over real bodies of text: source packs, transcripts, contracts, research, the public output of a person or company. Every ambition you have for that system — search, citation, reasoning, automation — passes through one narrow gate. Text becomes tokens, tokens become numbers, numbers go into the model, numbers come out, and numbers become text again.

If you understand that gate, three things stop being mysterious:

1. **Why context has limits.** You cannot paste an entire archive into a prompt. There is a hard ceiling, measured in tokens, and it shapes architecture.
2. **Why AI costs what it costs.** You are billed by the token, in both directions. Cost is not a vague function of "how much you use it" — it is a countable quantity you can estimate before you run anything.
3. **Why formatting changes behavior.** The model is sensitive to the *shape* of what you give it, not just the facts inside. A well-structured context and a wall of pasted noise can carry identical information and produce different results.

None of this is advanced. It is foundational, and most people building with AI skip it and then spend months confused by symptoms. We are going to spend one walk on the cause.

2. The core concept: a token is a chunk of text

Start with the plain definition and resist the urge to complicate it.

A token is a small chunk of text — usually a word, part of a word, or a punctuation mark — that the model treats as a single unit.

Tokens are not letters, and they are not always whole words. They sit in between. The English word *walking* might be one token. The word *tokenization* might be split into *token* + *ization*. A rare proper noun might shatter into four or five pieces. A space is usually part of the token that follows it, so *wa lk* (with the leading space) and *wa lk* are often *different* tokens.

A useful rule of thumb for English prose, accurate enough to plan with:

~4 characters per token. ~750 words per 1,000 tokens.

So a 1,500-word memo is roughly 2,000 tokens. A dense 300-page PDF can easily run past 150,000 tokens. Keep this conversion in your head; you will use it constantly.

Why chunks instead of whole words? Because whole words don't scale. The set of all possible English words — including names, typos, slang, code, URLs, and every language's vocabulary at once — is effectively infinite. A model needs a *fixed, finite* set of building blocks. Tokens are that compromise: small enough that a manageable inventory can reconstruct nearly any text, large enough that common words stay intact and efficient.

3. The mechanism underneath: tokenization and the vocabulary

Tokenization is the process of cutting text into tokens. The **vocabulary** is the fixed dictionary of every token the model knows — typically somewhere between 50,000 and 200,000 entries, depending on the model. Each entry maps to a number (an integer ID). Tokenization is, mechanically, "convert this string into a list of integers using the vocabulary."

The vocabulary is not handwritten. It is *learned* from a large corpus of text before the model is ever trained, using an algorithm — the most common family is **Byte-Pair Encoding (BPE)**. The intuition is worth understanding because it explains the model's blind spots.

How a vocabulary gets built (the short version)

BPE starts with the smallest possible units — individual characters or bytes — and then repeatedly merges the most *frequent adjacent pair* into a new single token. It does this thousands of times:

- Start: *t*, *h*, *e* are separate.
- *t+h* appear together constantly, so *th* becomes a token.
- *th+e* appear together constantly, so *the* becomes a token.
- ...and so on, until the vocabulary reaches its target size.

The result is elegant and has direct consequences:

- **Common words become single tokens** because they were merged early and often. *the*, *and*, *company* — one token each.
- **Rare words get split** into familiar sub-pieces. A surname the corpus rarely saw stays as fragments.
- **The tokenizer is frequency-shaped, not meaning-shaped.** It knows what *appeared together a lot*, not what *means* anything. This is the root of several failure modes in Section 7.

Three properties that matter operationally

Property	What it means	Why you care
Fixed vocabulary	The model only knows a finite set of tokens	Truly novel strings (long hashes, odd IDs) fragment into many tokens and cost more
Subword splitting	Words break into reusable pieces	Rare names, jargon, and code are less token-efficient than plain prose
Language & format bias	Vocab is shaped by its training corpus	English is cheap; many other languages, and messy formats, cost 2–4× more tokens for the same content

The single most useful thing you can do after reading this paper is open the **OpenAI Tokenizer** (link in the reading guide), paste in a paragraph of your own text, and *watch it split*. Paste a clean sentence. Then paste a Telegram export with timestamps and usernames. Then paste a table. The token count will surprise you, and that surprise is the lesson landing.

4. Context length: the size of the model's working memory

Once text is tokens, those tokens go *into* the model — but not unlimited tokens. Every model has a **context length** (also called the context window): the maximum number of tokens it can consider at once, counting **both** what you put in *and* what it generates back.

Think of it as the model's working memory for a single call. Not its knowledge — its knowledge lives in the trained weights. Context is the desk space available for *this specific task*. A model might "know" an enormous amount in general but can only look at what fits on the desk right now.

Two consequences follow immediately, and both are structural, not cosmetic:

Input and output share the budget. If your context window is 200,000 tokens and you fill 195,000 with input, you have left almost no room for a useful answer. Long inputs squeeze short answers. Budgeting context means budgeting *both directions*.

Anything past the limit cannot be considered — at all. This is the part people get wrong. The model does not "skim" or "remember the gist" of text beyond its window. Text outside the context window simply isn't there. It has no effect. There is no partial credit.

This is why "just give the model all our documents" is not a strategy. Your archive does not fit. It will never fit. The entire discipline of building useful AI systems over large knowledge — including everything GBrain is for — exists precisely *because* context is finite. You don't pour the ocean onto the desk. You decide what to bring.

The bridge to a later episode: the standard answer to "the data doesn't fit" is **RAG — retrieval augmented generation**. Instead of stuffing everything in, you store your documents externally, retrieve only the few most relevant chunks for a given question, and put *those* in the context window. We are not covering retrieval here. Just register that the entire need for it is created by the token ceiling you now understand.

5. Cost: you are billed by the token

Here is where this becomes a P&L line and not a curiosity.

You pay per token — for input and for output — on every single call.

Providers publish prices per million tokens, usually with **output tokens priced higher than input tokens** (generating is more expensive than reading). Because you can count tokens before you run anything, you can *estimate cost in advance*. This is rare and valuable: AI spend is one of the few software costs that is genuinely predictable per operation.

A worked example, using round numbers for intuition rather than any specific price sheet:

- You want to summarize a **300-page PDF** → roughly **150,000 input tokens**.
- You do this for **200 documents** → **30,000,000 input tokens**.
- The summaries themselves are short → say **500 output tokens each** → **100,000 output tokens** total.

The arithmetic is trivial once you have the token counts. The point is that the token counts are *knowable*, so the bill is *plannable*. A workflow's unit economics live and die on its token footprint. Two designs that produce the "same" result can differ 5× in cost depending on how much text each one shoves through the window and how often.

This reframes a lot of architecture decisions as **token-efficiency decisions**:

- Do you re-send the same background context on every call, or structure things so you don't?
- Do you summarize once and reuse the summary, or re-process raw text repeatedly?
- Do you feed the model a clean 2,000-token extract or a raw 40,000-token dump that happens to contain the same answer?

Same outcome, very different bill. Cost discipline *is* token discipline.

6. Compression and formatting: shape is information

Now the most strategically useful idea in this episode, and the one most people never internalize.

Two pieces of text can carry the same facts and cost wildly different numbers of tokens. The difference is structure, density, and noise. Getting more meaning into fewer tokens is, loosely, **compression** — and it is one of the highest-leverage skills in building with AI, because it directly buys you both *more room in the window* and *lower cost per call*, simultaneously.

Consider the same information in two forms:

- **Raw Telegram export:** every line carries a timestamp, a username, system notices ("X joined the group"), emoji, "ok", "lol", forwarded headers, link previews. The *signal* might be three decisions buried across 400 lines. You could spend 8,000 tokens to convey what matters in 300.
- **Cleaned summary:** "Decision 1: ... Decision 2: ... Open question: ..." — 300 tokens, higher signal, and far easier for the model to use well.

The same is true of long PDFs (headers, footers, page numbers, repeated boilerplate, OCR garble) and of source packs assembled by hand. Cleaning and structuring your context is not housekeeping. It is the act of fitting more truth onto a fixed-size desk for less money.

And there is a second, subtler effect beyond cost:

Format changes behavior, not just price. A model attends to structure. Clear sections, labels, and consistent formatting help it locate and use the right information. A noisy, unstructured blob can bury the relevant fact among ten thousand irrelevant tokens, and the answer degrades — even though, technically, "the information was in there."

"It was in the context" is not the same as "the model used it well." This is why two prompts containing identical facts can produce a sharp answer and a vague one. The well-formatted one made the relevant tokens easy to find and weighted; the messy one drowned them. When a result disappoints, your first question should not be "does the model know this?" but "did I make the relevant tokens easy to attend to, or did I bury them?"

7. The main limitation: tokenizers don't understand, they cut

Every failure mode in this section traces back to one fact from Section 3: **the tokenizer was built from frequency, not meaning**. It is a mechanical cutter, blind to semantics. That blindness shows up in predictable ways.

Character-level tasks are unreliable. Ask a model how many rs are in "strawberry" and it may stumble — not because it's stupid, but because it never saw the word as letters. It saw a couple of tokens. The internal letters were never individually present. Counting characters, reversing strings, and precise spelling are weak spots for exactly this reason.

Rare and structured strings explode. Long IDs, hashes, unusual names, and dense code fragment into many tokens. They cost more and the model handles them less reliably than common prose. If your workflow leans on exact long identifiers, you are working against the grain of the tokenizer.

Non-English and messy formats cost more. The same meaning in a language underrepresented in the vocabulary, or in a noisy export format, can take 2–4× the tokens. Same desk, less room, higher bill — purely from format.

The window is a hard wall, silently hit. Exceed the context length and content is dropped or rejected. Worse, even *within* the limit, models can underweight material buried in the middle of a very long context. Putting something in the window is necessary, not sufficient.

The honest summary: tokenization is a brilliant, lossy compromise that makes language models possible at all — and it leaks in specific, learnable places. Knowing where it leaks is most of what separates someone who builds robust AI workflows from someone who is mystified by intermittent failures.

8. Where this lands in FRIDAY and GBrain

Tie it together without overreaching.

FRIDAY's job is to operate over large bodies of text that will never fit in a single context window. That is not a flaw to engineer around someday; it is the permanent condition. **GBrain exists precisely because context is finite** — it is

the substrate that stores knowledge *outside* the window so the right, small, well-formatted slice can be brought *into* the window when a task needs it.

For your first wedge — making a person's or company's public body of work searchable, citable, and operationally useful — every principle in this paper is load-bearing:

- The corpus is far larger than any window, so you store and retrieve rather than paste. (Token ceiling → Section 4.)
- How you **clean and chunk** the source material — stripping boilerplate from PDFs, structuring transcripts — determines both quality and cost. (Compression → Section 6.)
- The economics of the product are, concretely, **token economics**. (Cost → Section 5.)
- Citations work only if the retrieved chunks are clean and well-bounded — another formatting discipline. (Format → Section 6.)

You do not need to build any of that this week. You need to be able to look at a proposed workflow and ask the right first question: *how many tokens, in which direction, formatted how, and does it fit?* That question, asked early, prevents most expensive mistakes.

Vocabulary

Token — A small chunk of text (a word, part of a word, or a punctuation mark) treated by the model as a single unit. Roughly 4 characters of English on average.

Tokenization — The process of cutting text into tokens and mapping each to a numeric ID using the vocabulary. The first step in turning language into computation.

Vocabulary — The fixed dictionary of all tokens a model knows (commonly 50k–200k entries), learned from a corpus before training. Frequency-shaped, not meaning-shaped.

Byte-Pair Encoding (BPE) — A common algorithm for building a vocabulary by repeatedly merging the most frequent adjacent pairs of units into single tokens. Why common words stay whole and rare words split.

Context length / context window — The maximum number of tokens a model can consider in one call, counting input *and* output together. The model's working memory for a single task.

Cost (per token) — Pricing model for running an LLM: billed per token of input and per token of output, with output usually priced higher. Predictable in advance because tokens are countable.

Compression — Conveying the same meaning in fewer tokens by removing noise and adding structure. Buys both window space and lower cost at once.

Prompt formatting — The structure and cleanliness of the text you put in context. Affects not only token count but how reliably the model uses the relevant information.

RAG (retrieval augmented generation) — Storing documents externally and retrieving only the most relevant chunks into the context window for a given question. The standard response to a finite context length. (Covered in a later episode; noted here only as the bridge.)

Reading guide

Keep this short and hands-on. Two anchors, both worth a few focused minutes — not a syllabus.

1. OpenAI Tokenizer — <https://platform.openai.com/tokenizer>

The single highest-value five minutes you can spend on this topic. Don't read it — *use* it. Paste in:

- a clean English sentence (watch words map to single tokens),
- a chunk of a raw Telegram or chat export (watch timestamps and usernames inflate the count),
- a long ID or hash (watch it shatter), and
- a small table or some code (watch the count balloon versus plain prose).

Goal: develop a physical intuition for *what costs tokens*. That intuition is the whole episode made tangible.

2. Hugging Face — Tokenizers Summary — https://huggingface.co/docs/transformers/tokenizer_summary

A clear technical overview of the main tokenization approaches, including BPE. Slightly more technical than this paper; that's the point. Read the conceptual sections to confirm the mental model in Section 3 — you do not need to follow every code example. Aim to come away able to say, in one sentence, *why* common words are single tokens and rare words are many.

If you only do one thing: open the tokenizer and paste your own text in. The rest will make more sense afterward.

Walking questions

Carry these on the next walk. They are for thinking, not for homework.

1. **What part of this could I explain cleanly right now** — to a smart person who has never thought about it? (Try the sentence: "A token is...")
2. **What still feels blurry?** Is it the vocabulary-building step, the shared input/output budget, or why formatting changes behavior?
3. **Where does this already show up in FRIDAY or GBrain** — and specifically, where is the context window the real constraint rather than model intelligence?
4. **What is one bounded, safe workflow** — small enough to fail cheaply — where I could test these ideas? (For instance: take one long PDF, clean it, and compare the token count and answer quality before and after.)

One-sentence takeaway

A model never sees your words — it sees tokens, and once you understand that text becomes a finite, countable, formattable stream of chunks, the limits, costs, and quirks of working with AI stop being mysteries and become design decisions you control.